



Hugging Face



Models



Datasets



Spaces



Community



Docs

Pricing



Log In

Sign Up

Join the Hugging Face community

and get access to the augmented documentation experience



Collaborate on models, datasets and Spaces



Faster examples with accelerated inference

[Sign Up](#)

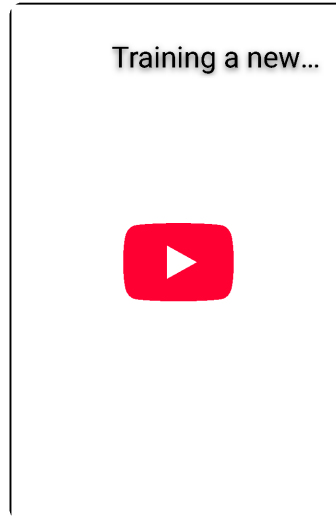
to get started

[Ask a question](#)[Open in Colab](#)[Open](#)[Studio Lab](#)

Training a new tokenizer from an old one

If a language model is not available in the language you are interested in, or if your corpus is very different from the one your language model was trained on, you will most likely want to retrain the model from scratch using a tokenizer adapted to your data. That will require training a new tokenizer on your dataset. But what exactly does that mean? When we first looked at tokenizers in [Chapter 2](#), we saw that most Transformer models use a *subword tokenization algorithm*. To identify which subwords are of interest and occur most frequently in the corpus at hand, the tokenizer needs to take a hard look at all the texts in the corpus — a process we call

training. The exact rules that govern this training depend on the type of tokenizer used, and we'll go over the three main algorithms later in this chapter.



⚠ Training a tokenizer is not the same as training a model! Model training uses stochastic gradient descent to make the loss a little bit smaller for each batch. It's randomized by nature (meaning you have to set some seeds to get the same results when doing the same training twice). Training a tokenizer is a statistical process that tries to identify which subwords are the best to pick for a given corpus, and the exact rules used to pick them depend on the tokenization algorithm. It's deterministic, meaning you always get the same results when training with the same algorithm on the same corpus.

Assembling a corpus

There's a very simple API in 🤗 Transformers that you can use to train a new tokenizer with the same characteristics as an existing one:

`AutoTokenizer.train_new_from_iterator()`. To see this in action, let's say we want to train GPT-2 from scratch, but in a language other than English. Our first task will be to gather lots of data in that language in a training corpus. To provide examples everyone will be able to understand, we won't use a language like Russian or Chinese here, but rather a specialized English language: Python code.

The 🧐 Datasets library can help us assemble a corpus of Python source code. We'll use the usual `load_dataset()` function to download and cache the CodeSearchNet dataset. This dataset was created for the CodeSearchNet challenge and contains millions of functions from open source libraries on GitHub in several programming languages. Here, we will load the Python part of this dataset:

```
from datasets import load_dataset

# This can take a few minutes to load
raw_datasets = load_dataset("code_s
```

We can have a look at the training split to see which columns we have access to:

```
raw_datasets["train"]
```

```
Dataset({
  features: ['repository_name', 'func_code_string', 'func_code_url'],
  num_rows: 412178
})
```

We can see the dataset separates docstrings from code and suggests a tokenization of both. Here, we'll just use the `whole_func_string` column to train our tokenizer. We can look at an example of one of these functions by indexing into the `train` split:

```
print(raw_datasets["train"][123456])
```

which should print the following:

```
def handle_simple_responses(  
    self, timeout_ms=None, info_cb=  
    """Accepts normal responses from the server.  
  
    Args:  
        timeout_ms: Timeout in milliseconds.  
        info_cb: Optional callback for progress information.  
  
    Returns:  
        OKAY packet's message.  
    """  
    return self._accept_responses('
```

The first thing we need to do is transform the dataset into an *iterator* of lists of texts — for instance, a list of list of texts. Using lists of texts will enable our tokenizer to go faster (training on batches of texts instead of processing individual texts one by one), and it should be an iterator if we want to avoid having everything in memory at once. If your corpus is huge, you will want to take advantage of the fact that 🤗 Datasets does not load everything into RAM but stores the elements of the dataset on disk.

Doing the following would create a list of lists of 1,000 texts each, but would load

everything in memory:

```
# Don't uncomment the following line  
# training_corpus = [raw_datasets["train"]]
```

Using a Python generator, we can avoid Python loading anything into memory until it's actually necessary. To create such a generator, you just need to replace the brackets with parentheses:

```
training_corpus = (  
    raw_datasets["train"][i : i + 10000]  
    for i in range(0, len(raw_datasets["train"]), 10000)  
)
```

This line of code doesn't fetch any elements of the dataset; it just creates an object you can use in a Python `for` loop. The texts will only be loaded when you need them (that is, when you're at the step of the `for` loop that requires them), and only 1,000 texts at a time will be loaded. This way you won't exhaust all your memory even if you are processing a huge dataset.

The problem with a generator object is that it can only be used once. So, instead of this giving us the list of the first 10 digits twice:

```
gen = (i for i in range(10))  
print(list(gen))  
print(list(gen))
```

we get them once and then an empty list:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
[]
```

That's why we define a function that returns a generator instead:

```
def get_training_corpus():
    return (
        raw_datasets["train"][i : i + 1024]
        for i in range(0, len(raw_datasets["train"]), 1024)
    )

training_corpus = get_training_corpus()
```

You can also define your generator inside a for loop by using the yield statement:

```
def get_training_corpus():
    dataset = raw_datasets["train"]
    for start_idx in range(0, len(dataset), 1024):
        samples = dataset[start_idx : start_idx + 1024]
        yield samples["whole_function"]
```

which will produce the exact same generator as before, but allows you to use more complex logic than you can in a list comprehension.

Training a new tokenizer

Now that we have our corpus in the form of an iterator of batches of texts, we are ready to train a new tokenizer. To do this, we first need to load the tokenizer we want to pair with our model (here, GPT-2):

```
from transformers import AutoTokenizer

old_tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Even though we are going to train a new

LLM Course ▾

EN ▾



🔄 3,564

0. SETUP

1. TRANSFORMER MODELS

2. USING 🤖 TRANSFORMERS

3. FINE-TUNING A PRETRAINED MODEL

4. SHARING MODELS AND TOKENIZERS

5. THE 🤖 DATASETS LIBRARY

6. THE 🤖 TOKENIZERS LIBRARY

Introduction

Training a new tokenizer from an old one

Fast tokenizers' special powers

Fast tokenizers in the QA pipeline

Training a new tokenizer from an old one

Assembling a corpus

Training a new tokenizer

Saving the tokenizer

Normalization and pre-tokenization
Byte-Pair Encoding tokenization
WordPiece tokenization
Unigram tokenization
Building a tokenizer, block by block
Tokenizers, check!
End-of-chapter quiz

7. CLASSICAL NLP TASKS

8. HOW TO ASK FOR HELP

9. BUILDING AND SHARING DEMOS

10. CURATE HIGH-QUALITY DATASETS

tokenizer, it's a good idea to do this to avoid starting entirely from scratch. This way, we won't have to specify anything about the tokenization algorithm or the special tokens we want to use; our new tokenizer will be exactly the same as GPT-2, and the only thing that will change is the vocabulary, which will be determined by the training on our corpus.

First let's have a look at how this tokenizer would treat an example function:

```
example = '''def add_numbers(a, b):  
    """Add the two numbers `a` and  
    return a + b'''
```

```
tokens = old_tokenizer.tokenize(example)  
tokens
```

```
['def', 'Ġadd', 'Ġ_', 'Ġn', 'Ġumbers',  
 'Ġnumbers', 'Ġ`', 'Ġa', 'Ġ'', 'Ġand']
```

This tokenizer has a few special symbols, like `Ġ` and `Ċ`, which denote spaces and newlines, respectively. As we can see, this is not too efficient: the tokenizer returns individual tokens for each space, when it could group together indentation levels (since having sets of four or eight spaces is going to be very common in code). It also split the function name a bit weirdly, not being used to seeing words with the `_` character.

Let's train a new tokenizer and see if it solves those issues. For this, we'll use the method `train_new_from_iterator()`:

```
tokenizer = old_tokenizer.train_new
```

This command might take a bit of time if your corpus is very large, but for this dataset of 1.6 GB of texts it's blazing fast (1 minute 16 seconds on an AMD Ryzen 9 3900X CPU with 12 cores).

Note that

`AutoTokenizer.train_new_from_iterator()` only works if the tokenizer you are using is a “fast” tokenizer. As you'll see in the next section, the 🧠 Transformers library contains two types of tokenizers: some are written purely in Python and others (the fast ones) are backed by the 🧠 Tokenizers library, which is written in the Rust programming language. Python is the language most often used for data science and deep learning applications, but when anything needs to be parallelized to be fast, it has to be written in another language. For instance, the matrix multiplications that are at the core of the model computation are written in CUDA, an optimized C library for GPUs.

Training a brand new tokenizer in pure Python would be excruciatingly slow, which is why we developed the 🧠 Tokenizers library. Note that just as you didn't have to learn the CUDA language to be able to execute your model on a batch of inputs on a GPU, you won't need to learn Rust to use a fast tokenizer. The 🧠 Tokenizers library provides Python bindings for many methods that internally call some piece of code in Rust; for example, to parallelize the training

of your new tokenizer or, as we saw in [Chapter 3](#), the tokenization of a batch of inputs.

Most of the Transformer models have a fast tokenizer available (there are some exceptions that you can check [here](#)), and the `AutoTokenizer` API always selects the fast tokenizer for you if it's available. In the next section we'll take a look at some of the other special features fast tokenizers have, which will be really useful for tasks like token classification and question answering. Before diving into that, however, let's try our brand new tokenizer on the previous example:

```
tokens = tokenizer.tokenize(example  
tokens
```

```
['def', 'Ġadd', 'Ġ_', 'numbers', 'Ġ(',  
'a', 'Ġ', 'Ġand', 'Ġ`', 'b', 'Ġ'.']
```

Here we again see the special symbols `Ġ` and `Ġ` that denote spaces and newlines, but we can also see that our tokenizer learned some tokens that are highly specific to a corpus of Python functions: for example, there is a `ĠĠĠ` token that represents an indentation, and a `Ġ"""` token that represents the three quotes that start a docstring. The tokenizer also correctly split the function name on `_`. This is quite a compact representation; comparatively, using the plain English tokenizer on the same example will give us a longer sentence:

```
print(len(tokens))
print(len(old_tokenizer.tokenize(ex
```

```
27
36
```

Let's look at another example:

```
example = """class LinearLayer():
    def __init__(self, input_size,
                  self.weight = torch.randn(i
                  self.bias = torch.zeros(out

    def __call__(self, x):
        return x @ self.weights + s
    """
tokenizer.tokenize(example)
```

```
['class', 'ĠLinear', 'Layer', '():',
 'Ġoutput', '_', 'size', '):', 'ĠĠĠ
 'size', ',', 'Ġoutput', '_', 'size
 'output', '_', 'size', ')', 'ĠĠĠĠĠ
 'Ġreturn', 'Ġx', 'Ġ@', 'Ġself', '.']
```

In addition to the token corresponding to an indentation, here we can also see a token for a double indentation: ĠĠĠĠĠĠĠ. The special Python words like `class`, `init`, `call`, `self`, and `return` are each tokenized as one token, and we can see that as well as splitting on `_` and `.` the tokenizer correctly splits even camel-cased names:

`LinearLayer` is tokenized as `["ĠLinear", "Layer"]`.

Saving the tokenizer

To make sure we can use it later, we need to save our new tokenizer. Like for models, this

is done with the `save_pretrained()` method:

```
tokenizer.save_pretrained("code-sea
```

This will create a new folder named `code-search-net-tokenizer`, which will contain all the files the tokenizer needs to be reloaded. If you want to share this tokenizer with your colleagues and friends, you can upload it to the Hub by logging into your account. If you're working in a notebook, there's a convenience function to help you with this:

```
from huggingface_hub import notebook_login\n\nnotebook_login()
```

This will display a widget where you can enter your Hugging Face login credentials. If you aren't working in a notebook, just type the following line in your terminal:

```
huggingface-cli login
```

Once you've logged in, you can push your tokenizer by executing the following command:

```
tokenizer.push_to_hub("code-search-
```

This will create a new repository in your namespace with the name `code-search-net-tokenizer`, containing the tokenizer file. You can then load the tokenizer from anywhere with the `from_pretrained()` method:

```
# Replace "huggingface-course" below  
tokenizer = AutoTokenizer.from_pretrained("huggingface-course/llm-course-tokenizer")
```

You're now all set for training a language model from scratch and fine-tuning it on your task at hand! We'll get to that in [Chapter 7](#), but first, in the rest of this chapter we'll take a closer look at fast tokenizers and explore in detail what actually happens when we call the method `train_new_from_iterator()`.

</> [Update on GitHub](#)

← Introduction

Fast tokenizers' special powers →